I102 - Paradigmas de Programación

OOP – SOBRECARGA DE OPERADORES

5

Prof. Mariano Scaramal

Ingeniería en Inteligencia Artificial

Esta Presentación Cubre…

- Conceptos:
  - Los calificadores *const* y *static* aplicados a clases.
  - Sobrecarga de operadores:
    - Ejemplo con operador lógico.
    - Ejemplo con operador aritmético.
  - Funciones y clases friend.
  - Copiar Objetos
    - Shallow Copy
    - Deep Copy

Métodos, Atributos y Objetos Tipo *const*

- Los **métodos** *const* no pueden realizar cambios en el objeto. Esto se logra:
  - No pudiendo cambiar valores de atributos dentro del método,
  - No pudiendo llamar métodos que no sean *const*, dado que estos sí podrían cambiar atributos del objeto.
- Los **atributos** o variables miembro *const*, deben seguir las siguientes reglas:
  - Deben ser inicializados cuando el objeto es creado,
  - No podrán volver a cambiar dado que son calificados como *inmutables*.
  - Sólo las variables ***static const*** pueden ser inicializadas (no las *static*).
- Los **objetos** *const* no pueden ser modificados:
  - Los atributos son declarados en el constructor,
  - Sólo se pueden llamar métodos tipo *const*, forzado por el compilador.

Métodos, Atributos y Objetos Tipo *const* (cont'd)

Como estas reglas pueden ser demasiado estrictas, existe el calificador *mutable* que permite, por ejemplo, modificar un atributo con un método constante.

Así, *counter* puede ser modificado por un método *const*.

Esto puede hacerse incluso habiendo definido el objeto como *const*.

```
int MyClass::sInt = 8;

int main() {
   const MyClass obj(10);     // Const object
   obj.incrementCounter();  // Permitido
   obj.incrementCounter();  // Permitido
   std::cout << "Counter: " << obj.getCounter()
                 << std::endl;  // Imprime: Counter: 12
   // obj.sInt  no puede ser accedido, es private
   return 0;
}
```

```
class MyClass {
private:
   mutable int counter;   // Atributo mutable
   const int cInt;        // Se debe inizialiar en el constructor
   const int cIInt = 3;   // Permitido
   static int sInt;       // Se debe definir fuera de la clase
   //static int sIInt = 5;  // No está permitido
   static const int y = 4;  // Permitido

public:
   MyClass() : counter(0) , cInt(3) {}
   MyClass(int val) : counter(val) , cInt(3) {}

   void incrementCounter() const {
      counter++;  // Permitido porque 'counter' es mutable
   }

   int getCounter() const { return counter; }
};
```

Atributos Tipo *static*

Los atributos *static* son variables que pueden ser compartidos entre todas las instancias de la clase (es decir, son atributos de clase).

La inicialización se hace fuera de la clase aunque son declarados dentro.

```
#include <iostream>

class MyClass {
private:
   static int counter;  // Esta variable de clase
                        //debe inicializarse.
   static const int PI = 3.14159; // Se debe definir

public:
   void incrementCounter() {
      counter++;
   }

   static int getCounter() {
      return counter;
   }
};
```

```
// Debe hacerse la Inicializacion del atributo static
fuera de la clase
int MyClass::counter = 0;   // No es necesario
instanciar una clase para acceder la variable

int main() {
   MyClass obj; // Major definer static en constructor
   obj.incrementCounter(); obj.incrementCounter();
   std::cout << "Counter: " << obj.getCounter()
             << std::endl;  // Output: Counter: 2

   MyClass obj2; obj2.incrementCounter();
   std::cout << "Counter: " << obj2.getCounter()
             << std::endl;  // Imprime: Counter: 3

   return 0;
}
```

Métodos Tipo *static*

Los métodos static pertenecen a la clase y no a instancias particulares. Sólo pueden acceder directamente a atributos y métodos también declarados static.

Su uso es común en librerías matemática, donde se reciben valores como entrada para realizar cálculos y donde las constantes suelen ser tipo static.

```
#include <iostream>
#include <cmath>  // Para calcular la exponencial

class Math {
public:
   // Constante tipo static
   static const double SQRT_2PI;

   // Metodo static para  calcular la distribucion Gaussiana
   static double gaussian(double x, double mean, double stddev) {
      double exponent = -((x - mean) * (x - mean)) /
                                                (2 * stddev * stddev);
      return (1 / (stddev * SQRT_2PI)) * std::exp(exponent);
   }
};
```

```
const double Math::SQRT_2PI = 2.5066282746310002;

int main() {
   double x = 0.0;              // Punto a evaluar
   double mean = 0.0;           // Valor medio
   double stddev = 1.0;         // Desvio standard

   // Se usa el metodo estático para calcular el valor
   std::cout << "El valor de la gaussiana de x = " << x << " es "
             << Math::gaussian(x, mean, stddev)
             << " (o 1/SQRT_2PI)" << std::endl;
   // Valor de la gaussiana en x = 0 es 0.398942 (o 1/SQRT_2PI)
   return 0;
}
```

Sobrecarga de Operadores

En el caso de tener distintas instancias de una clase con distintos datos, surgen las siguientes pregunta:

1. ¿Cómo comparar, sumar, restar o realizar alguna otra operación con estos objetos?
2. ¿Tiene algún sentido comparar dos objetos? ¿Cómo se comparan 2 satélites?

Comencemos respondiendo la pregunta 2. La respuesta se puede obtener pensando en que sus nombre (tipo *string)* se concatenan mediante la operación “+”. Pero, en realidad, la operación puede ser definida según la necesidad del usuario.

La primer pregunta es más corta de responder: utilizando sobrecarga de operadores.

```
class X
{
    int i;
    X& operator+(const X &r)
    {
        i=i-r.i;
        return *this;
    }
};
```

Sobrecarga de Operadores

En C++, es posible redefinir el comportamiento de los operadores para que actúen sobre objetos de clases definidas por el usuario. Esto se conoce como sobrecarga de operadores.

Los operadores que se sobrecargan suelen ser los más comunes:

==, !=, <, >, =, +

La sobrecarga de operadores se logra mediante la adición de un método o función, solo que su nombre es ***operator*** seguido del operador a sobrecargar. Por ejemplo:

<tipoDeRetorno> operator <símbolo> (<argumentos>);

Tipo de retorno

Operador a sobrecargar

Referencia constante al objeto

Sobrecarga de Operador Lógico – Ejemplo 1

En la sobrecarga del operador binario “>”, el parámetro del método representa al operando derecho (car2), mientras que el operando izquierdo corresponde al objeto actual (this).

```
class Car {
private:
    std::string maker;
    double length;  // Largo del auto en centímetros

public:
    Car(std::string maker, double length) : maker(maker),
                                            length(length) {}

    // Sobrecargando el operador '>' para comparar por length
    bool operator>(const Car& other) const {
        return this->length > other.length;
    }
};
```

```
int main() {
    Car car1("Toyota", 450.0);
    Car car2("Ford", 400.0);

    if (car1 > car2)
        std::cout << "Car 1 es mas largo que car 2\n";
    else
        std::cout << "Car 1 no es mas largo que car 2\n";
    // Imprime: Car 1 es mas largo que car 2
    return 0;
}
```

Sobrecarga de Operador Lógico – Ejemplo 2

La comparación se realiza de manera análoga. Es importante notar que dos objetos pueden ser considerados iguales sin ser el mismo objeto. El criterio de igualdad lo define el desarrollador.

```
#include <iostream>
#include <string>

class Car {
private:
   std::string maker; int year;

public:
   Car(std::string maker, int year) : maker(maker), year(year) {}

   // Sobrecarga de '==' para ver si los autos son iguales
   bool operator==(const Car& other) const {
      return (this->maker == other.maker) &&
             (this->year == other.year);
   }
};
```

```
int main() {
   Car car1("Toyota", 2010); Car car2("Toyota", 2010);
   Car car3("Ford", 2010);

   // Comparar car1 y car2
   if (car1 == car2) // Imprime: Car1 y car 2 son iguales.
      std::cout << "Car 1 y car 2 son iguales.\n";
   else
      std::cout << "Car 1 y car 2 son diferentes.\n";
   // Comparar car1 y car3
   if (car1 == car3) // Imprime: Car1 y car 3 son diferentes.
      std::cout << "Car 1 y car 3 son iguales.\n";
   else
      std::cout << "Car 1 y car 3 son diferentes.\n";

   return 0;
}
```

Sobrecarga de Operador Aritmético – Ejemplo 1

De manera análoga, el operador “+” puede sobrecargarse para definir una operación específica, como el cálculo de la longitud total de una flota de autos.

```
#include <iostream>
#include <string>

class Car {
private:
   std::string maker;
   double length;  // Largo del auto en centimetros

public:
   Car(std::string maker, double length) : maker(maker), length(length) {}

   // Sobrecarga de'+' para sumar las longitudes de los autos
   double operator+(const Car& other) const {
      return this->length + other.length;
   } // El segundo operando, car2, es pasado como parámetero
};
```

```
int main() {
   Car car1("Toyota", 450.0);
   Car car2("Ford", 400.0);

   // Imprime la suma de las longitudes de los autos
   std::cout << "La longitud total de los autos es: "
             << car1 + car2 << " cm\n";

   return 0;
}
```

Sobrecarga de Operador Aritmético – Ejemplo 2

```
#include <iostream>
#include <string>

class Car {
private:
   std::string maker; double length;  // Largo del auto

public:
   Car(std::string maker, double length) : maker(maker),
   length(length) {}

   // Sobrecarga '+=' para obtener un vehículo mas largo
   Car& operator+=(double extraLength) {
      this->length += extraLength;     // Agrega largo extra
      return *this;          // Devuelve el objeto modificado
   }

   void display() const { // Muestra la información
      std::cout << "Fabricante: " << make << ", Largo: " <<
      length << " cm\n";
   }
};
```

Se retorna *Car*&, porque se desea tener concatenación para esta operación.

<u>En general</u>, las operaciones que modifican el objeto (+, +=, *, -, etc), se retornan por referencia (&). Las operaciones que no modifican el objeto (==, >, !=, etc), se retorna por valor.

```
int main() {
   Car car("Toyota", 450.0);
   // Agrega 50 cm y luego 30 cm a la longitud original del auto
   (car += 50) += 30;
   car.display();  // Muestra que la longitud del auto es 530 cm

   return 0;
}
```

Sobrecarga con Funciones No Miembro

No es necesario que la sobrecarga de un operador sea un método de una clase; también puede definirse como una función no miembro.

El compilador selecciona la sobrecarga adecuada en función del nombre de la función y de los tipos de sus parámetros.

```
#include <iostream>
using namespace std;

class Plane {
public:
    double wingspan;
    Plane(double w) : wingspan(w) {}
};
```

```
// Sobrecarga del operador +
Plane operator+(const Plane& a, const Plane& b) {
    return Plane(a.wingspan + b.wingspan);
}

int main() {
    Plane p1(30.0), p2(35.0);
    Plane total = p1 + p2; // Llama a operator+(p1, p2)

    return 0;
}
```

Sobrecarga

- Usar funciones miembro si:
  - Actúan directamente sobre la memoria del objeto. Ejemplo: Operadores de Asignación (=, +=, -=, *=) o unarios (++, –).
  - Ventaja: Tienen acceso directo a los atributos private.

- Usar funciones NO miembro si:
  - Se utilizan principalmente para operadores binarios donde se busca simetría (por ejemplo, si queres permitir que tanto *objeto* + *5* como *5* + *objeto* funcionen).
  - El operando de la izquierda no es de tu clase (por ejemplo << (cout) y >> (cin)).

Funciones *friend*

Las funciones *friend* son funciones que no forman parte de un objeto (no son un método) pero que pueden tener acceso a miembros *public*, private y *protected* de la clase. No obstante, no tiene acceso a *"this".*

La declaración de la función *friend* se hace dentro de la clase, pero su declaración se realiza fuera de la misma.

Debido a esto, las funciones friends son denominadas "funciones no miembro", que no pertenece a una clase.

La "amistad" no es mutua, es decir, si un objeto tiene una función *friend*, la función puede acceder los miembros *public*, *private* y *protected*, pero la clase no puede acceder a ninguna variable de la función.

Funciones *friend* - Ejemplo

- La declaración se hace dentro de la clase y su definición por fuera.
- La función tiene acceso a los atributos *private* oilStatus y *private waterStatus*.
- Usar estas funciones con precaución para no romper el encapsulamiento.

```
class Plane {
private:
   double oilStatus;
   double waterStatus;
public:
   Plane(double oil, double water):
         oilStatus(oil), waterStatus(water){}

   void setOilStatus(double oil){oilStatus = oil;};

   void setWaterStatus(double water){
       waterStatus = water;
   };

   friend void showPlaneData(Plane pl);
};
```

```
void showPlaneData(Plane pl){
   std::cout << "The oil status is: " << pl.oilStatus
                << ", the water status is: " << pl.waterStatus
                << std::endl;
}

int main(){
   Plane myPlane(75, 100);
   showPlaneData(myPlane);
   // Imprime: Plane oil is: 75, Plane water is: 100

   return 0;
}
```

Clases *friend*

De manera análoga, una clase friend es una clase cuyos métodos pueden acceder a los miembros public, private y protected de otra clase que la declara como friend.

En este caso, la declaración de friend se realiza a nivel de clase, es decir, se declara una clase como friend, no un objeto en particular.

Al igual que ocurre con las funciones friend, la “amistad” no es mutua. Si una clase declara a otra como friend, esta última puede acceder a sus miembros, pero no ocurre lo mismo en sentido inverso.

No obstante, dos clases pueden declararse mutuamente friend si se desea acceso en ambas direcciones.

Clases *friend* - Ejemplo

```
class Car;

class Engine {
private:
   double horsepower;
   bool isOn;
   void displayEngine() {
      std::cout << "Power: " << horsepower
                << ", Engine on? "
                << ( isOn ? "Yes" : "No" )
                << std::endl;
   }

public:
   Engine(double horsepower, bool isOn) :
         horsepower(horsepower),isOn(isOn) {}

   // Se hace amigo de la classe Engine
   friend class Car;
};
```

```
class Car {
private:
   std::string maker;  double length;
public:
   Car(std::string maker, double length) : maker(maker),
                                           length(length) {}

   void displayInfo(Engine eg){
      std::cout << "Car maker: " << maker << ", Length: "
                << length << std::endl << "Engine data: ";
      eg.displayEngine();  // Método privado de la clase Engine
      std::cout << "Engine is: " << (eg.isOn ? "Yes" : "No")
                << std::endl;
   }
};

int main() {
   Car car("Toyota", 4.5);
   Engine eng(300, true); // Un motor de 300 hp activado
   car.displayInfo(eng);

   return 0;
}
```

Sobrecarga de Operadores con *friend*

Se sobrecarga el operador “<<“ que recibe una clase ostream que pertenece a la C++ Standard Library (parte de iostream), para imprimir los datos del objeto Plane en una manera particular.

```
#include <iostream>
#include <string>
using namespace std;

class Plane {
   string model;
   int capacity;
   double wingspan;

public:
   Plane(string m, int c, double w) :
         model(m), capacity(c), wingspan(w) {}

   friend ostream& operator<<(ostream& os, const
   Plane& p); // Amistad con función no miembro
};
```

```
// Esta es una función no miembro, no es parte de una clase
ostream& operator<<(ostream& os, const Plane& p) {
    os << "Plane Model: " << p.model << "\n"
       << "Capacity: " << p.capacity << " passengers\n"
       << "Wingspan: " << p.wingspan << " meters \n";
   return os;
}

int main() {
   Plane plane("Boeing 747", 416, 68.4);
   // Usa el operador “<<“ para imprimir el objeto
   cout << plane; // Idem a hacer: operator<<(cout, plane)
   return 0;
}
```

Sobrecarga con Funciones No Miembro y No *Friend*

Universidad de SanAndrés

Se sobrecarga el operador “<<“ que recibe una clase ostream que pertenece a la C++ Standard Library (parte de iostream), para imprimir los datos del objeto Plane, sin ser ‘friend’.

```
#include <iostream>
#include <string>
using namespace std;

class Plane {
   string model;
   int capacity;
   double wingspan;
public:
   Plane(string m, int c, double w) :
         model(m), capacity(c), wingspan(w) {}
// Getters públicos (necesarios si no usamos friend)
string getModel() const { return model; }
int getCapacity() const { return capacity; }
};
```

```
// Esta es una función no miembro, no es parte de una clase
ostream& operator<<(ostream& os, const Plane& p) {
    os << "Plane Model: " << p.getModel() << "\n"
       << "Capacity: " << p.getCapacity() << " passengers\n";
   return os;
}

int main() {
   Plane plane("Boeing 747", 416, 68.4);
   // Usa el operador “<<“ para imprimir el objeto
   cout << plane; // Idem a hacer: operator<<(cout, plane)
   return 0;
}
```

Copiar Objetos – Shallow Copy

Cuando se copia un objeto en C++ (ya sea mediante el operador = o el constructor de copia), el compilador genera por defecto una copia miembro a miembro.

1. myClass c1(3, 5);
2. myClass c2;
3. c2 = c1;

Este comportamiento se denomina ***Shallow Copy***.

Una *shallow copy* es adecuada cuando los miembros del objeto no gestionan memoria dinámica. Sin embargo, si el objeto contiene punteros a memoria alocada dinámicamente, sólo se copia la dirección, lo que puede provocar problemas como doble liberación de memoria o *dangling pointers*.

Copiar Objetos – Deep Copy

Cuando un objeto contiene punteros a memoria alocada dinámicamente, es necesario implementar una deep copy.

Una deep copy crea un nuevo objeto con su propia memoria, copiando el contenido en lugar de la dirección de los punteros.

En C++, este comportamiento puede implementarse mediante el copy constructor y el operador de asignación. Por defecto, ambos realizan una shallow copy, pero pueden redefinirse para realizar una deep copy.

Alternativamente, el uso de smart pointers permite gestionar la copia de manera más segura y, en muchos casos, evitar la necesidad de implementar una deep copy manualmente.

Copy Constructor

Un *copy constructor* es un constructor especial que se utiliza para crear un nuevo objeto como copia de otro existente.

Con este tipo de constructor, el objeto se copia al ser creado:

```
MyClass obj1(10);
MyClass obj2 = obj1;
```

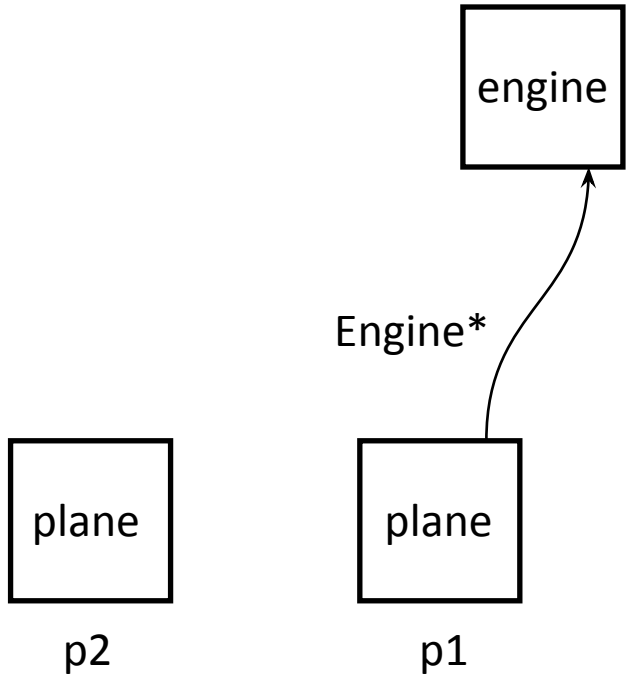
En este caso, la copia del objeto se realiza mediante el copy constructor, por lo que no interviene el operador de asignación (=).

Para ello, se define un constructor que recibe un objeto del mismo tipo (MyClass) y copia sus atributos.

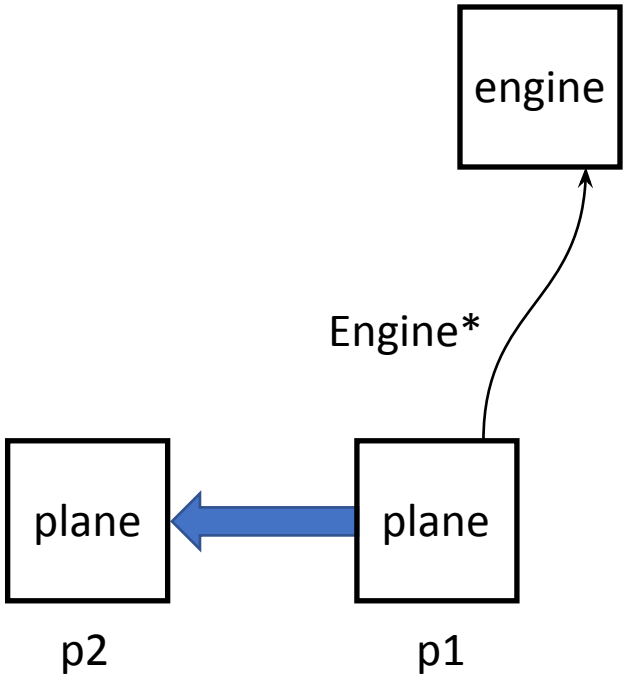
Este mecanismo puede utilizarse tanto para realizar una shallow copy como una deep copy, según la implementación.

Copy Constructor – Shallow Copy

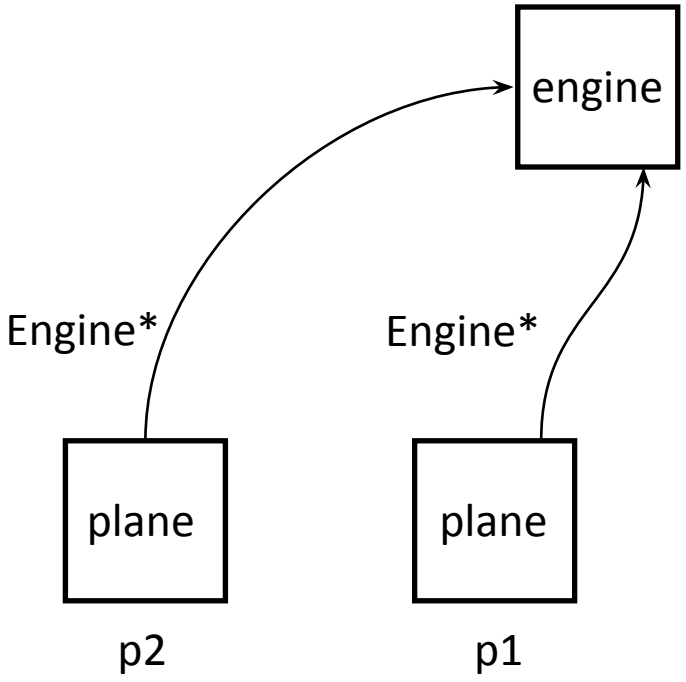
Se puede ver el problema en la siguiente secuencia de eventos:



Se crea el objeto p2, en el que se hará la copia del objeto p1.



Se copia la dirección del puntero Engine de p1 a p2.



Los punteros Engine de p1 y p2 apuntan a la misma dirección.

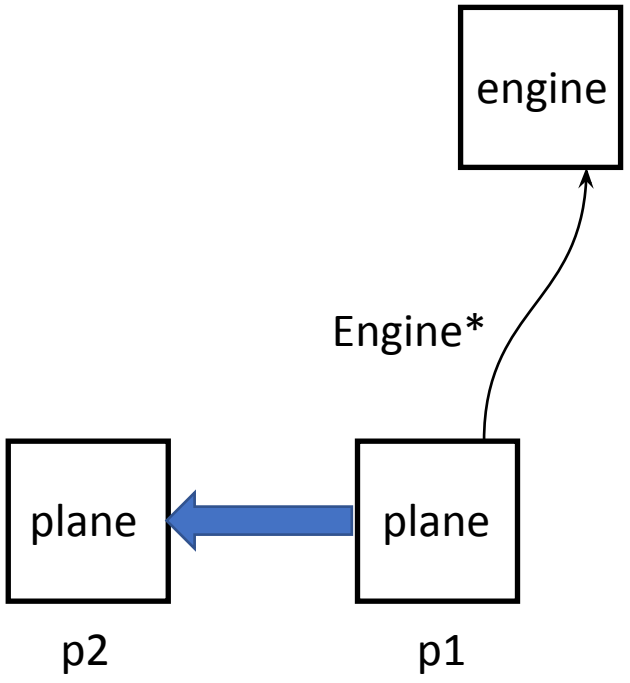
Copy Constructor – Ejemplo de Shallow Copy

En general no tiene sentido escribir un copy constructor para hacer una *Shallow Copy* dado que es el default del compilador. Este ejemplo ilustra la necesidad de *Deep Copy*.

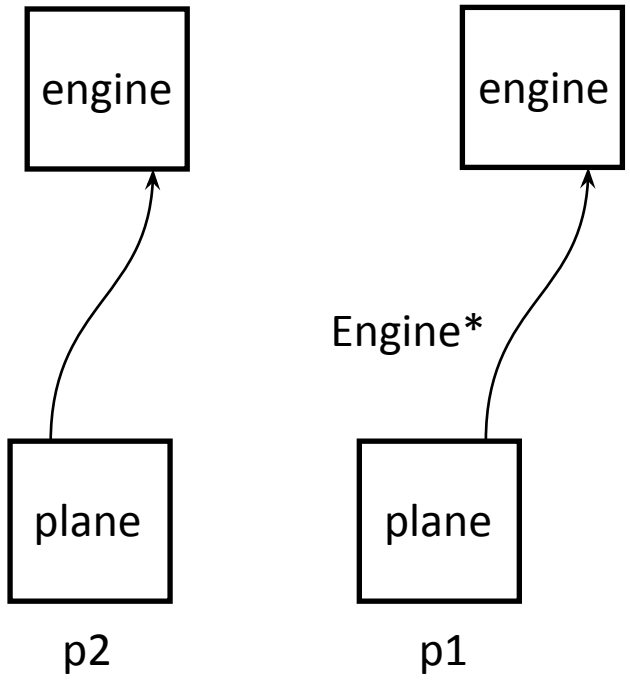
```
class Engine {
public:
   Engine(int power) : power(power) {}
   int getPower() const {return power; }
private:
   int power;
};

class Plane {
public:
    //  Constructor con entero
   Plane(int power) : engine(new Engine(power)) {}
   // Constructor que copia el puntero (Shallow Copy)
   Plane(const Plane& rht) : engine(rht.engine) {}

   void display() const {
      std::cout << "Engine Power: " << engine->getPower()
                << "\n";
   }

   ~Plane() {
      delete engine;
   // Se borrara el puntero. Si este destructor lo
   //llama primero p2 se producirá un error porque
   //puntero a Engine ya se liberó y no existe en p1.
   }
private:
   Engine* engine;
};

int main() {
   Plane p1(200);
   Plane p2 = p1; // Shallow Copy

   p1.display(); // Ambos comparten el
   p2.display(); // mismo puntero a Engine

   return 0;
}
```

Copy Constructor – Deep Copy

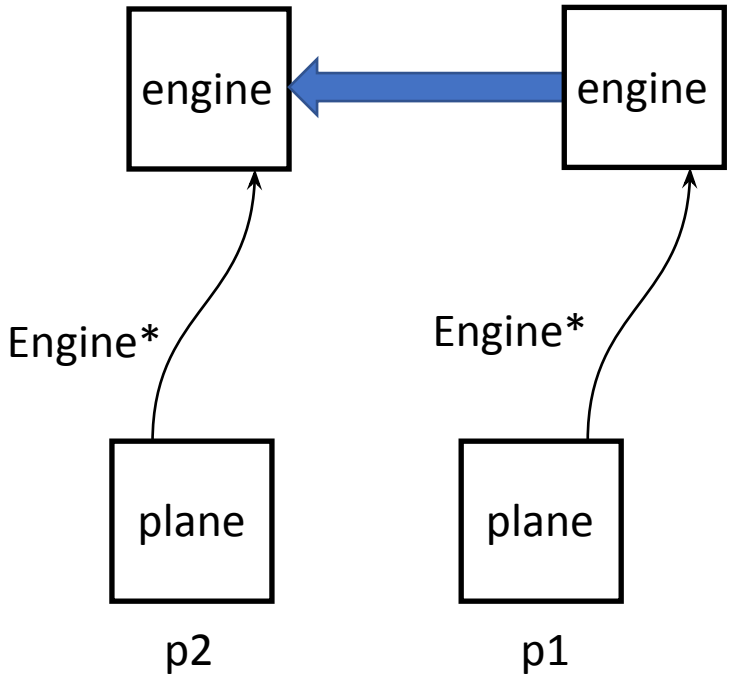
Se puede ver el problema en la siguiente secuencia de eventos:



Se crea el objeto p2, en el que se hará la copia del objeto p1.



Se crea un nuevo puntero a un objeto Engine para p2.



Se copia el objeto Engine de p1 a la posición de memoria de p2.

Copy Constructor – Ejemplo de Deep Copy

```
class Engine {
public:
   Engine(int power) : power(power) {}
   int getPower() const {return power;}
private:
   int power;
};

class Plane {
public:
   Plane(int power) : engine(new Engine(power)) {}
   // Constructor tipo Deep Copy
   Plane(const Plane& rht) : engine( new Engine(
                                    (rht.engine)->getPower()  ) ){}
   // En la linea anterior se creo un nuevo puntero a Engine
   void display() const {
      std::cout << "Engine Power: " << engine->getPower()
                << "\n";
    }
```

```
   ~Plane() { delete engine; }
private:
   Engine* engine;
};

int main() {
   Plane p1(200);
   Plane p2 = p1; // Al Deep Copy Constructor
   // Los objetos Engine son independientes
   p1.display(); p2.display();

   return 0;
}
// Como los objetos Engine son distintos
objetos en distintas posiciones de memoria,
son independientes.
```

Copiar Objetos – Deep Copy

```
#include <iostream>
#include <memory> // Para los smart pointers

class Engine {
public:
   Engine(int power) : power(power) {}
   int getPower() const { return power; }
private:
   int power;
};

class Plane {
public:
   Plane(int serial, int length, int power)
      : iSerialNumber(serial), iLength(length),
      engine(std::make_unique<Engine>(power)) {}
   // Deep Copy con Copy Constructor usando unique_ptr
   Plane(const Plane& rht) // Deep copy de engine
      : iSerialNumber(rht.iSerialNumber),
       iLength(rht.iLength),
       engine(std::make_unique<Engine>(
                                rht.engine->getPower() )  ) {}
```

```
 void display() const {
      std::cout << "Serial Number: " << iSerialNumber << "\n";
      std::cout << "Length: " << iLength << "\n";
      std::cout << "Engine Power: " << engine->getPower() << "\n";
   }
private:
   int iSerialNumber;
   int iLength;
   std::unique_ptr<Engine> engine; // unique pointer para Engine
};

int main() {
   Plane p1(1234, 50, 200);
   Plane p2 = p1; // Deep copy usando copy constructor

   std::cout << "Plane 1 Details:\n";
   p1.display();
   std::cout << "\nPlane 2 Details (after deep copy):\n";
   p2.display();

   return 0;
} // No hay necesidad de escribir los destroyers
```

Copia con Sobrecarga de Operador=

Se sobrecarga el operador “=“ para realizar una *deep copy* al asignar un objeto a otro.

Antes de copiar, se libera la memoria existente para evitar *memory leaks*, y luego se reserva nueva memoria copiando el contenido.

```
#include <iostream>
using namespace std;

class A {
public:
    int* data;
    A(int value) { data = new int(value); }

    // Operador de asignación (deep copy)
    A& operator=(const A& other) {
        if (this == &other) return *this; // Auto-asignación

        delete data; // Liberar memoria actual
        data = new int(*other.data); // Copiar contenido

        return *this;
    }
};
```

```
void imprimirData(A a, A b){
    cout << "a.x: " << *(a.data) << ", "
         << "b.x: " << *(b.data) << endl;
}

int main(){
    int x = 23;
    A a(x), b(5);

    imprimirData(a, b); // a.x: 23, a.y: 5
    b = a;
    imprimirData(a, b); // a.x: 23, a.y: 23

    return 0;
}
```

Preguntas?